

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

ОТЧЁТ

Домашняя работа 3

Тестирование API средствами Postman

Выполнил:
Прель Александр
Группа БР1.1

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Реализовать тестирование API в Postman и написать проверки внутри коллекции. Подготовить один комплексный сценарий основного процесса приложения для бронирования столиков в ресторанах.

Ход работы

1. Исходные материалы и среда

Использованы OpenAPI из Д32, работающий монолит ЛР1 и микросервисная версия ЛР2. На macOS arm64 установлены Node.js 24.14.0, npm 11.9.0. Консольный прогон выполнен Newman 6.2.2. Дата проверки: 09.09.2026.

Коллекция restaurant-booking-dz3.postman_collection.json и окружение restaurant-booking-local.postman_environment.json находятся в homeworks/hw3. Первоначальные 16 запросов дополнены 13 запросами с ошибочными действиями и проверкой прав.

2. Комплексный пользовательский сценарий

Запросы	Действия	Проверки
1-5	Health, регистрация, вход, профиль, изменение имени	HTTP-коды, JWT, email, сохранение имени
6-11	Кухни, поиск и фильтрация, карточка ресторана, меню, фото, отзывы	Структура ответа, ID, список результатов и пагинация
12-16	Создание отзыва и брони, история, детали брони, отмена	201, наличие брони в истории, confirmed -> cancelled
17-19	Нет токена, неправильный пароль, повторный email	401 UNAUTHORIZED, 409 CONFLICT
20-24	Неверные поля, отсутствующий ресторан, нет мест, повторная отмена	422, 404, 409, сохранённое состояние cancelled
25-29	Второй пользователь, чужая бронь, собственная история	403 FORBIDDEN, изоляция данных, доступ владельца

У каждого ответа проверяются JSON Content-Type и отсутствие passwordHash. Проверка ожидаемого ответа 403 считается успешной, если сервер действительно запрещает доступ.

3. Передача состояния между запросами

Перед регистрацией генерируется уникальный email с UUID и очищаются переменные старого запуска. После входа сохраняется accessToken, после поиска restaurantId, после создания брони reservationId. Второй JWT хранится отдельно в secondaryToken. Токены не включены в исходные файлы и отчёт.

```
Пример теста:
pm.test('Бронь создана', () => pm.response.to.have.status(201));
pm.test('Статус confirmed', () => {
  pm.expect(pm.response.json().status).to.eql('confirmed');
});
```

4. Воспроизведение

В labs/lab2 выполнить `npm ci --include=dev`, `npm run setup`, `npm run build`, `npm start`. В другом терминале из той же папки выполнить `npm run test:postman`. Для ручного запуска импортировать коллекцию и окружение в Postman, установить `baseUrl` и запустить всю коллекцию по порядку.

ЛР2 использует Gateway `http://127.0.0.1:8080`. Для сравнения ЛР1 была запущена на временном порту 61968 и временной SQLite-базе. После проверки временный монолит остановлен.

Исходные данные ЛР1 не изменялись.

5. Фактические результаты

Приложение	Requests	Assertions	Failed
ЛР2: микросервисы	29	117	0
ЛР1: исходный монолит	29	117	0

Тесты создают двух пользователей, отзыв и отменённую бронь. Они не очищают существующие записи, поэтому нужен отдельный тестовый набор баз. При прерывании до отмены может остаться активная бронь.

Вывод

Подготовлен комплексный сценарий из 29 запросов и 117 проверок. Он успешно выполнен на монолите и микросервисах. Проверены основной пользовательский процесс, формат ответов, обработка ошибок и изоляция бронирований пользователей.

Подробные команды: `homeworks/hw3/README.md`. Результаты: `labs/lab2/TEST_RESULTS.md`. Защита и принятие преподавателем не являются частью автоматической проверки.